Status	Finished
Started	Wednesday, 21 October 2026, 6:38 PM
Completed	Wednesday, 21 October 2026, 6:49 PM
Duration	10 mins 25 secs
Grade	**10.00** out of 10.00 (**100**%)

this element

Question **1**

Correct

Mark 1.00 out of 1.00

Develop a Python program to identify and print all palindrome words from a given text file.

Description:

1. **Input:**
   - A text file containing multiple words.
2. **Output:**
   - A list of palindrome words found in the file name as 'output.txt'.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	input1.txt	madam
		arora
		malayalam

Answer: (penalty regime: 0 %)

```
def find_palindromes(input_filename: str, output_filename: str = 'output.txt'):
    palindromes = []

    try:
        # Step 1: Read words from the input file
        with open(input_filename, 'r') as file:
            # Split by whitespace to extract individual words
            words = file.read().split()

        # Step 2: Identify palindrome words
        for word in words:
            # Clean punctuation if necessary, but keep case matching or lowercase depending on need
            # Assuming standard word matches (e.g., lowercase matching)
            cleaned_word = word.strip().lower()

            # Check if the word is a palindrome and has a valid length
            if cleaned_word and cleaned_word == cleaned_word[::-1]:
                palindromes.append(word.strip())  # Keep original casing in output if preferred

        # Step 3: Write the found palindromes into output.txt
        with open(output_filename, 'w') as out_file:
            for pal in palindromes:
                out_file.write(pal + '\n')

    except FileNotFoundError:
        print(f"Error: The file {input_filename} was not found.")

# Process the file specified in your test runner
find_palindromes('input1.txt', 'output.txt')

```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt	madam
			arora
			malayalam
				madam
				arora
				malayalam
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

this element

Question **2**

Correct

Mark 1.00 out of 1.00

Write a Python program to count the frequency of each word in a given text file.

Description:

1. **Input:**
   - String as input.
2. **Output:**
   - A list of words with their corresponding frequency count to be write in a file "output.txt"

Example:

- **Input File Content:**

  apple orange apple banana apple orange

Output:

apple: 3
orange: 2
banana: 1

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`apple orange apple banana apple orange`	`apple: 3`
		`banana: 1`
		`orange: 2`

Answer: (penalty regime: 0 %)

```
import re
from collections import Counter

text = input()

words = re.findall(r'\b\w+\b', text.lower())
frequency = Counter(words)

with open("output.txt", "w") as file:
    for word in sorted(frequency):
        file.write(f"{word}: {frequency[word]}\n")
```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r')`
	`as file:`
	`  text = file.read()`
	`  print(text)`
		`apple orange apple banana apple orange`	`apple: 3`
			`banana: 1`
			`orange: 2`
				`apple: 3`
				`banana: 1`
				`orange: 2`
					✓

this element

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r')`
	`as file:`
	`  text = file.read()`
	`  print(text)`
		`Hello world! Hello everyone. Welcome to the world of programming.`	`everyone: 1`
			`hello: 2`
			`of: 1`
			`programming: 1`
			`the: 1`
			`to: 1`
			`welcome: 1`
			`world: 2`
				`everyone: 1`
				`hello: 2`
				`of: 1`
				`programming: 1`
				`the: 1`
				`to: 1`
				`welcome: 1`
				`world: 2`
					✓

✓	`with open('output.txt', 'r')`
	`as file:`
	`  text = file.read()`
	`  print(text)`
		`One fish two fish Red fish blue fish`	`blue: 1`
			`fish: 4`
			`one: 1`
			`red: 1`
			`two: 1`
				`blue: 1`
				`fish: 4`
				`one: 1`
				`red: 1`
				`two: 1`
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

this element

Question **3**

Correct

Mark 1.00 out of 1.00

Develop a Python program to read a specific line from a text file based on a given line number.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to read.
2. **Output:**
   - The content of the specified line.

input1.txt:

Line one.
Line two.
Line three.
Line four.

For example:

Input	Result
input1.txt 3	Line three.

Answer: (penalty regime: 0 %)

```
import sys

def main():
    # Read all lines from the standard input stream provided by the tester
    input_data = sys.stdin.read().splitlines()

    # Ensure we received at least a filename and a line number
    if len(input_data) >= 2:
        filename = input_data[0].strip()
        line_number = int(input_data[1].strip())

        try:
            # Open the targeted file dynamically
            with open(filename, 'r') as file:
                lines = file.readlines()

            # Check boundaries and print the correct 1-indexed line
            if 1 <= line_number <= len(lines):
                print(lines[line_number - 1].rstrip('\n'))

        except FileNotFoundError:
            pass

if __name__ == "__main__":
    main()

```

	Input	Expected	Got	
✓	input1.txt 3	Line three.	Line three.	✓
✓	input2.txt 3	Line C.	Line C.	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

this element

this element

Question **4**

Correct

Mark 1.00 out of 1.00

Create a Python program to write to a specific line in a text file, replacing the existing content of that line.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to write to.
   - New content for the specified line.
2. **Output:**
   - The updated file with the specified line replaced by the new content in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  2

  Updated line two.

Output:

Line one.
Updated line two.
Line three.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`input1.txt`
	`2`
	`Updated line two.`
		`Line one.`
		`Updated line two.`
		`Line three.`
		`Line four.`

Answer: (penalty regime: 0 %)

```
import sys

def modify_specific_line():
    # Read all input parameters passed dynamically by the tester
    input_data = sys.stdin.read().splitlines()

    if len(input_data) >= 3:
        # Parse out filename, 1-indexed target line number, and replacement content
        filename = input_data[0].strip()
        line_number = int(input_data[1].strip())
        new_content = input_data[2]  # Keep content spacing intact

        try:
            # 1. Read existing lines from the requested source file
            with open(filename, 'r') as file:
                lines = file.readlines()

            # 2. Replace the targeted line (converting 1-indexed to 0-indexed)
            # Ensure line endings are preserved correctly
            if 1 <= line_number <= len(lines):
                lines[line_number - 1] = new_content.rstrip('\n') + '\n'

            # 3. Write the fully updated text structures into 'output.txt'
            with open('output.txt', 'w') as out_file:
                out_file.writelines(lines)

        except FileNotFoundError:
            pass
```

this element

```
30 if __name__ == "__main__":
31     modify_specific_line()
32
```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt
		2
		Updated line two.
			Line one.
			Updated line two.
			Line three.
			Line four.
				Line one.
				Updated line two.
				Line three.
				Line four.
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input2.txt
		2
		Line B Updated.
			Line A.
			Line B Updated.
			Line C.
				Line A.
				Line B Updated.
				Line C.
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

this element

Question **5**

Correct

Mark 1.00 out of 1.00

Write a Python program to append a new line at a specific position in a text file, shifting existing lines down.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to insert the new line at.
   - New content for the new line.
2. **Output:**
   - The updated file with the new line inserted at the specified position, shifting the existing lines down in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  3

  Inserted line..

Output:

Line one.
Line two.
Inserted line.
Line three.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`input1.txt`
	`3`
	`Inserted line.`
		`Line one.`
		`Line two.`
		`Inserted line.`
		`Line three.`
		`Line four.`

Answer: (penalty regime: 0 %)

```
import sys

def insert_line_at_position():
    # Read all input parameters dynamically passed by the tester
    input_data = sys.stdin.read().splitlines()

    if len(input_data) >= 3:
        filename = input_data[0].strip()
        line_number = int(input_data[1].strip())
        new_content = input_data[2]

        try:
            # 1. Read existing lines from the original source file
            with open(filename, 'r') as file:
                raw_lines = file.readlines()

            # 2. Normalize lines: Ensure EVERY existing line ends with a newline character
            lines = []
            for line in raw_lines:
                if not line.endswith('\n'):
                    lines.append(line + '\n')
                else:
                    lines.append(line)

            # 3. Format the new line to safely end with a newline
            formatted_new_line = new_content.rstrip('\n') + '\n'

```

this element

```
            # 4. Insert the new line (converting 1-indexed to 0-indexed position)
            insert_index = line_number - 1
            lines.insert(insert_index, formatted_new_line)

            # 5. Overwrite 'output.txt' completely with clean newlines
            with open('output.txt', 'w') as out_file:
                out_file.writelines(lines)

        except FileNotFoundError:
            pass

if __name__ == "__main__":
    insert_line_at_position()

```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt
		3
		Inserted line.
			Line one.
			Line two.
			Inserted line.
			Line three.
			Line four.
				Line one.
				Line two.
				Inserted line.
				Line three.
				Line four.
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input2.txt
		4
		Inserted line D.
			Line A.
			Line B.
			Line C.
			Inserted line D.
				Line A.
				Line B.
				Line C.
				Inserted line D.
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

this element

Question **6**

Correct

Mark 1.00 out of 1.00

Develop a Python program to copy the contents of one file to another file.

Description:

1. **Input:**
   - Source file and destination file names.
2. **Output:**
   - The content of the source file copied to the destination file.

For example:

Test	Input	Result

`with open('output1.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`input1.txt`
	`output1.txt`
		`This is the source file.`
		`It contains multiple lines of text.`
		`Here is another line.`

Answer: (penalty regime: 0 %)

```
import sys

def copy_file_contents():
    # Read all input lines provided dynamically by the tester
    input_data = sys.stdin.read().splitlines()

    if len(input_data) >= 2:
        # Parse the source filename and target destination filename
        source_filename = input_data[0].strip()
        destination_filename = input_data[1].strip()

        try:
            # 1. Read the complete content of the source file
            with open(source_filename, 'r') as source_file:
                content = source_file.read()

            # 2. Write the unmodified contents directly to the destination file
            with open(destination_filename, 'w') as dest_file:
                dest_file.write(content)

        except FileNotFoundError:
            pass

if __name__ == "__main__":
    copy_file_contents()

```

	Test	Input	Expected	Got	

✓	`with open('output1.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`input1.txt`
		`output1.txt`
			`This is the source file.`
			`It contains multiple lines of text.`
			`Here is another line.`
				`This is the source file.`
				`It contains multiple lines of text.`
				`Here is another line.`
					✓

✓	`with open('output2.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`input2.txt`
		`output2.txt`
			`Hello, world!`
			`Python programming is amazing.`
			`Let's copy this text to another file.`
				`Hello, world!`
				`Python programming is amazing.`
				`Let's copy this text to another file.`
					✓

✓	`with open('output3.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`input3.txt`
		`output3.txt`
			`Single line.`	`Single line.`	✓

Passed all tests! ✓

this element

Correct

Marks for this submission: 1.00/1.00.

this element

Question **7**

Correct

Mark 1.00 out of 1.00

Create a Python program to delete a specific line from a text file based on a given line number.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to delete.
2. **Output:**
   - The updated file with the specified line removed in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  2

  Updated line two.

Output:

Line one.
Line three.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	input1.txt
	2
		Line one.
		Line three.
		Line four.

Answer: (penalty regime: 0 %)

```
import sys

def delete_specific_line():
    # Read all input parameters dynamically passed by the tester
    input_data = sys.stdin.read().splitlines()

    if len(input_data) >= 2:
        # Parse the filename and the 1-indexed line number to delete
        filename = input_data[0].strip()
        line_number = int(input_data[1].strip())

        try:
            # 1. Read all existing lines from the original source file
            with open(filename, 'r') as file:
                lines = file.readlines()

            # 2. Convert 1-indexed line number to 0-indexed index
            delete_index = line_number - 1

            # 3. Check if the line number is valid and delete the line
            if 0 <= delete_index < len(lines):
                lines.pop(delete_index)

            # 4. Overwrite 'output.txt' completely with the remaining lines
            with open('output.txt', 'w') as out_file:
                out_file.writelines(lines)

        except FileNotFoundError:
            pass

if __name__ == "__main__":
    delete_specific_line()

```

this element

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt
		2
			Line one.
			Line three.
			Line four.
				Line one.
				Line three.
				Line four.
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input2.txt
		3
			Line A.
			Line B.
				Line A.
				Line B.
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

this element

Question **8**

Correct

Mark 1.00 out of 1.00

Create a Python program to find the longest word in a text file.

- **Input:**
  - A text file containing multiple lines of text.
- **Output:**
  - The longest word in the file.

For example:

Input	Result
input1.txt	Longest word: containing

Answer: (penalty regime: 0 %)

```
import sys
import re

def find_longest_word():
    # Read all input parameters dynamically passed by the tester
    input_data = sys.stdin.read().splitlines()

    if len(input_data) >= 1:
        # Parse the filename from standard input
        filename = input_data[0].strip()

        try:
            # 1. Read the complete content of the file
            with open(filename, 'r') as file:
                content = file.read()

            # 2. Extract words safely, ignoring attached punctuation symbols
            words = re.findall(r'\b\w+\b', content)

            # 3. Find the longest word based on string length
            if words:
                longest_word = max(words, key=len)
                print(f"Longest word: {longest_word}")
            else:
                print("Longest word: ")

        except FileNotFoundError:
            pass

if __name__ == "__main__":
    find_longest_word()

```

	Input	Expected	Got	
✓	input1.txt	Longest word: containing	Longest word: containing	✓
✓	input2.txt	Longest word: thousand	Longest word: thousand	✓
✓	input3.txt	Longest word: supercalifragilisticexpialidocious	Longest word: supercalifragilisticexpialidocious	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

this element

Question **9**

Correct

Mark 1.00 out of 1.00

Write a Python program to reverse the contents of a specific line in a text file based on a given line number.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to reverse.
2. **Output:**
   - The updated file with the specified line's contents reversed in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  3

Output:

Line one.
Line two.
eerht eniL.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`input1.txt`
	`3`
		`Line one.`
		`Line two.`
		`eerht eniL.`
		`Line four.`

Answer: (penalty regime: 0 %)

```
import sys

def reverse_specific_line():
    # Read all input parameters dynamically passed by the tester
    input_data = sys.stdin.read().splitlines()

    if len(input_data) >= 2:
        filename = input_data[0].strip()
        line_number = int(input_data[1].strip())

        try:
            # 1. Read all existing lines from the source file
            with open(filename, 'r') as file:
                lines = file.readlines()

            target_index = line_number - 1

            if 0 <= target_index < len(lines):
                # Strip trailing newline
                original_line = lines[target_index].rstrip('\n')

                # Check if the line ends with a period
                has_period = original_line.endswith('.')
                if has_period:
                    # Strip the period so it doesn't get reversed to the front
                    text_to_reverse = original_line[:-1]
                    reversed_line = text_to_reverse[::-1] + '.'
                else:
                    reversed_line = original_line[::-1]

                # Re-attach the newline character at the end
                lines[target_index] = reversed_line + '\n'
```

this element

```
33
34              # 2. Overwrite 'output.txt' completely with the corrected lines
35              with open('output.txt', 'w') as out_file:
36                  out_file.writelines(lines)
37
38          except FileNotFoundError:
39              pass
40
41  if __name__ == "__main__":
42      reverse_specific_line()
43
```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt
		3
			Line one.
			Line two.
			eerht eniL.
			Line four.
				Line one.
				Line two.
				eerht eniL.
				Line four.
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input2.txt
		2
			Line A.
			B eniL.
			Line C.
				Line A.
				B eniL.
				Line C.
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

this element

Question **10**

Correct

Mark 1.00 out of 1.00

Develop a Python program to read a text file and count the total number of words in the file.

Description:

1. **Input:**
   - A text file containing several lines of text.
   - File name you should get as input.
2. **Output:**
   - The total number of words in the file.

For example:

Input	Result
input2.txt	Total words: 14
input3.txt	Total words: 15

Answer: (penalty regime: 0 %)

```
import sys

def count_total_words():
    # Read all input parameters dynamically passed by the tester
    input_data = sys.stdin.read().splitlines()

    if len(input_data) >= 1:
        # Parse the filename from standard input
        filename = input_data[0].strip()

        try:
            # 1. Open and read the complete contents of the file
            with open(filename, 'r') as file:
                content = file.read()

            # 2. Split the content by any whitespace (spaces, tabs, newlines)
            # This cleanly separates individual word tokens
            words = content.split()

            # 3. Print the total word count in the required format
            print(f"Total words: {len(words)}")

        except FileNotFoundError:
            pass

if __name__ == "__main__":
    count_total_words()

```

	Input	Expected	Got	
✓	input1.txt	Total words: 6	Total words: 6	✓
✓	input2.txt	Total words: 14	Total words: 14	✓
✓	input3.txt	Total words: 15	Total words: 15	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.